

CASE STUDY 3

Browser-Based Location Disclosure: A Controlled Seeker Link Simulation Case Study

By: Joseph Gonzalez



Scope	Controlled lab simulation using a researcher-owned phone posing as the victim device; educational and analytical purposes only
Environment	Kali Linux, Seeker, ngrok tunnel, mobile browser, researcher-owned Android phone, redacted public URL/IP/location artifacts
Primary Evidence	Seeker launch screenshot, ngrok tunnel screenshot, request listener screenshot, mobile location-permission screenshot, redacted terminal capture of device/location fields

Educational and analytical use only. Public URL, IP, device, and location details shown in figures are redacted.

Introduction

Mobile browsers routinely ask for sensitive permissions during ordinary web activity, especially when a site claims it needs location to personalize the experience. That creates a different kind of exposure from credential phishing or malicious app sideloading: the user does not need to install software or type a password. The risk appears at the permission prompt, where a believable page can persuade the user to share location data.

This case study examines a single controlled Seeker simulation using a temporary public tunnel and a researcher-owned phone posing as the victim device. The goal was to document what the page looked like, how the browser presented the location request, what data appeared in the terminal after permission was granted, and why this matters for individual users outside the lab.

Methodology

A controlled cybersecurity simulation was conducted in an isolated lab environment using Kali Linux, Seeker, an ngrok tunnel, and a mobile browser on a researcher-owned Android phone. The scope was limited to self-controlled infrastructure, a temporary test link, and consent-based interaction by the researcher. No real victims, third-party devices, production accounts, or unauthorized tracking were involved. Public URL, IP, device, and location artifacts were redacted before publication.

1. Overview

Seeker is useful as a defensive teaching tool because it shows how a web page can request location access and record browser provided device and network details once the user grants permission. In this scenario, the analytical question was whether a believable “near you” style page, delivered through a temporary public tunnel, could produce observable location and device data from a phone that voluntarily accepted the browser prompt.

2. Lab Setup

The lab host was used to start the Seeker environment and select a location themed template. As depicted in Figure 1, the available templates were presented from the terminal before the test page was staged. As depicted in Figure 2, a temporary tunnel was then used to make the local test page reachable from the phone. The significance of this stage is not the exact command sequence, but the transition from a local-only web page to a public link that a mobile browser could open.

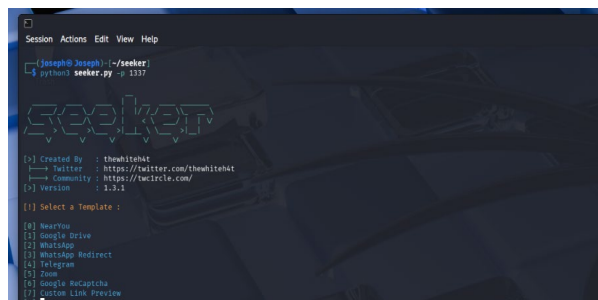


Figure 1. Seeker launch environment on the lab host, showing the template selection stage used for the controlled simulation.

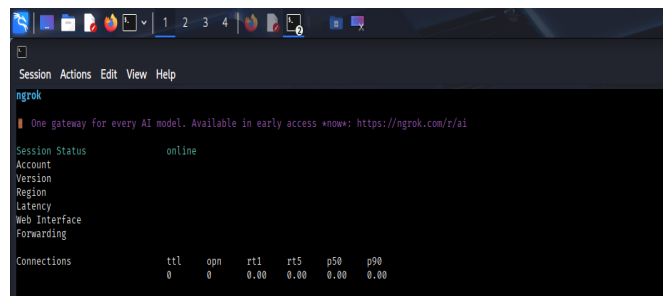


Figure 2. Temporary tunnel session used to expose the local test page through a public forwarding path, with URL and account details redacted.

3. Public Link Delivery and Permission Prompt

After the public test link was prepared, the phone opened the location-themed page as the simulated victim device. As depicted in Figure 3, request activity appeared in the tunnel session after the page was visited. As depicted in Figure 4, the mobile browser displayed a location permission request for the site. This is the key decision point: the exposure depends on the user granting access, not on an invisible bypass of the browser permission model.

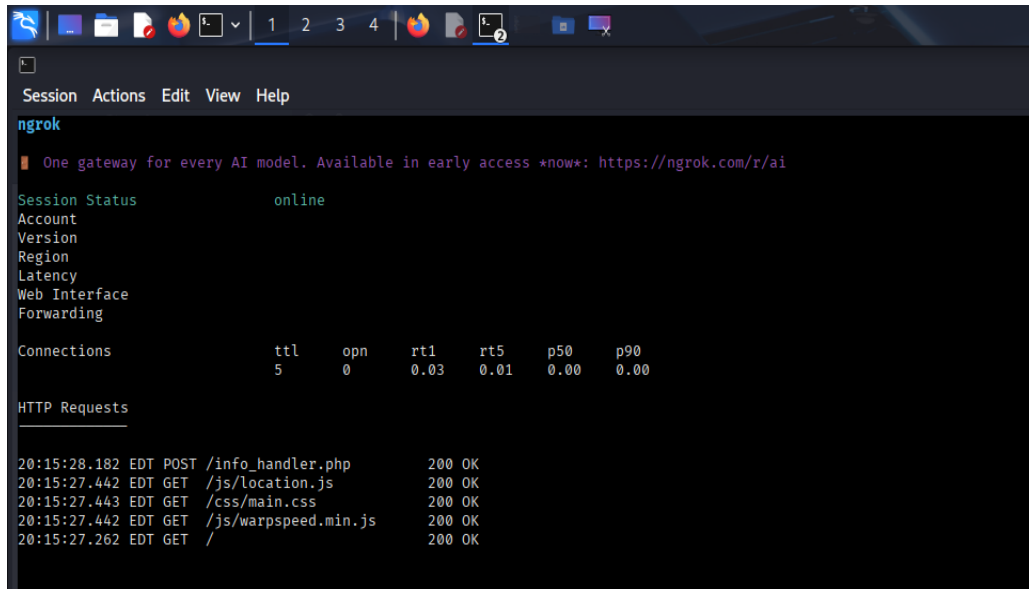


Figure 3. Tunnel listener view showing web requests from the phone after the public test link was opened, with forwarding details redacted.

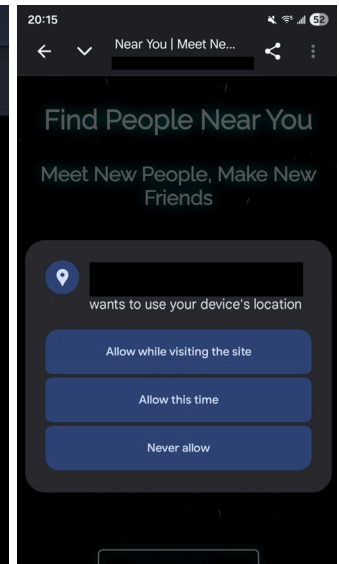


Figure 4. Researcher-owned phone posing as the victim device, opening the link and receiving a browser location-permission prompt with the public URL redacted.

4. Evidence Collected

As depicted in Figure 5, the terminal output recorded device, browser, network, and location-related fields after the phone allowed location access. The most important evidentiary point is that the browser prompt created a consent gate: once permission was granted, sensitive context was returned to the listening page and became visible in the collection terminal. The redactions preserve the forensic value of the artifact while removing private IP, URL, and location data.

```

(joseph@joseph) ~/seeker
$ python3 seeker.py -p 1337

[+] Created By : thewhiteh
    Twitter : https://t
    Community : https://t
    Version : 1.3.1

[!] Select a Template :
[0] NearYou
[1] Google Drive
[2] WhatsApp
[3] WhatsApp Redirect
[4] Telegram
[5] Zoom
[6] Google ReCaptcha
[7] Custom Link Preview
[>] 0

[+] Loading NearYou Template ...

[+] Port : 1337

[+] Starting PHP Server ... [ ✖ ]
[!] Old instance of php server found, restarting ...
[+] Starting PHP Server ... [ ✔ ]

[+] Waiting for Client ... [ctrl+c to exit]

[!] Device Information :
[+] OS : Android 10
[+] Platform : Linux armv81
[+] CPU Cores : 8
[+] RAM : 8
[+] GPU Vendor : Qualcomm
[+] GPU : Adreno (TM) 740
[+] Resolution : 384x824
[+] Browser
[+] Public IP

[!] IP Information :
[+] Continent
[+] Country
[+] Region
[+] City
[+] Org
[+] ISP

[!] Location Information :
[+] Latitude
[+] Longitude
[+] Accuracy
[+] Altitude
[+] Direction
[+] Speed

[+] Google Map
[+] Data Saved

[+] Waiting for

```

Figure 5. Redacted Seeker terminal capture showing browser-provided device, network, and location fields after permission was granted during the controlled test.

5. Why the Disclosure Occurred

The disclosure occurred because the site asked for a permission that browsers are designed to support, and the user granted it. In other words, this was not a silent operating-system compromise. It was a consent-driven exposure created by a convincing pretext, a public link, and a permission prompt that looked routine enough to approve.

The “near you” theme made the request feel contextually appropriate: a page claiming to show nearby people naturally appears to need location access. The browser did show the site origin, but users often focus on the task language and the large approval buttons rather than verifying the temporary domain. The tunnel also mattered because it turned a local service into a reachable public page, allowing the phone to interact with the lab-hosted content from outside the localhost context.

6. Real-World Impact to the Individual

For an individual user, granting location to a deceptive page can expose more than a single data point. Location can reveal where someone lives, works, studies, worships, or spends time with family. When combined with public IP address, device type, browser version, and network provider information, it can support targeted phishing, harassment, doxxing, stalking, or social-engineering attempts that feel unusually personal.

The risk is especially serious on phones because people treat mobile permission prompts as routine. A user may approve location while distracted, then forget that the site ever asked. Even if the page does not install malware, the collected data can still help an attacker build a profile of the person, time future messages, or craft a more believable follow-up lure.

7. Mitigation, Hardening & Prevention

The practical fix is to reduce unnecessary location grants and treat permission prompts as security decisions. Users should deny location requests from unfamiliar links, avoid opening location-themed links from messages or social media, and check the browser address bar before approving access. On phones, browser settings should be reviewed regularly so old site permissions can be removed.

If a suspicious location prompt was approved, the user should revoke the site permission, clear the site data, close the tab, and avoid interacting with the page again. For long-term prevention, keep the mobile browser and operating system updated, prefer official apps and known domains, disable precise location when it is not needed, and teach users that “Allow this time” still releases sensitive data for that session. A VPN may hide some network information, but it does not prevent GPS or browser-granted location disclosure once permission is accepted.

8. Ethical Consideration

This case study was conducted strictly for educational and analytical purposes. The site, tunnel, terminal session, and phone interaction were operated by the researcher in a bounded test scenario. The phone used in the screenshots was intentionally acting as the victim device, and all sensitive URLs, IP details, and location values were redacted before the report was prepared.

The value of the case is defensive rather than operational. It demonstrates how permission-based data exposure can happen without malware installation and without credential entry, simply because the user is persuaded to approve a browser request. That makes the case useful for awareness training, family device safety, mobile privacy discussions, and incident-response planning.

Ultimately, the lesson is straightforward: browser permission prompts are not harmless pop-ups. They are trust decisions. When a site asks for location, the safest habit is to pause, verify the origin, and deny the request unless the need is legitimate and expected.